

BLBW

Be Life, Be Water

MANUAL TÉCNICO AVANÇADO DE INSTRUÇÕES E LÓGICA DE CONTROLE

Referência Rápida e Aplicações Práticas - Siemens TIA Portal / Step 7 (LAD/STL)

Versão: Corporativa 1.0

Público-Alvo: Engenharia de Automação e Manutenção Industrial (Nível Especialista)

Plataforma: CLP Siemens (S7-1200, S7-1500, S7-300/400)

Data: 03/07/2026

ÍNDICE GERAL

CAPÍTULO 1: Bit Logic Operations

- 1.1 Contatos de Alta Performance e Bobinas de Atribuição
- 1.2 Memorização de Estado e Flip-Flops (SR / RS)
- 1.3 Operações de Detecção de Borda (P_TRIG / N_TRIG / R_TRIG / F_TRIG)

CAPÍTULO 2: Timer Operations (IEC Timers)

- 2.1 TON (Timer On-Delay)
- 2.2 TOF (Timer Off-Delay)
- 2.3 TP (Timer Pulse)
- 2.4 TONR (Time Accumulator)

CAPÍTULO 3: Counter Operations (IEC Counters)

- 3.1 CTU (Count Up)
- 3.2 CTD (Count Down)
- 3.3 CTUD (Count Up/Down)

CAPÍTULO 4: Comparator Operations

- 4.1 Operações Matemáticas Relacionais (==, <>, >=, <=, >, <)
- 4.2 Checagem de Limites (IN_Range / OUT_Range)

CAPÍTULO 5: Math Functions

- 5.1 Cálculos Aritméticos Básicos (ADD, SUB, MUL, DIV)
- 5.2 Seleção de Extremos e Limitação (MIN, MAX, LIMIT)
- 5.3 Funções Trigonométricas e Exponenciais Avançadas

CAPÍTULO 6: Move Operations

- 6.1 Transferência de Bloco Escalar (MOVE)
- 6.2 Manipulação em Lote e Serialização (MOVE_BLK, FILL_BLK, SWAP)

CAPÍTULO 7: Conversion Operations

- 7.1 Conversão Explícita de Cast (CONVERT)
- 7.2 Arredondamento e Truncamento (ROUND, CEIL, FLOOR, TRUNC)
- 7.3 Escalonamento de Sinais Analógicos (SCALE_X, NORM_X)

CAPÍTULO 8: Program Control Operations

- 8.1 Saltos e Roteamento Lógico (JMP, JMPN, LABEL, SWITCH)
- 8.2 Finalização de Bloco (RET)

CAPÍTULO 9: Word Logic Operations

- 9.1 Mascaramento a Nível de Palavra (AND, OR, XOR)
- 9.2 Manipulação Estrutural (ENCO, DECO, SEL, MUX, DEMUX)

CAPÍTULO 10: Shift and Rotate Operations

- 10.1 Deslocamento de Bits (SHR, SHL)
- 10.2 Rotação de Registradores (ROR, ROL)

CAPÍTULO 1: Bit Logic Operations

1.1 Contatos de Alta Performance e Bobinas de Atribuição

Contato Normalmente Aberto (NA) — ||

Nome e Objetivo: Elemento de entrada que conduz corrente lógica (RLO = 1) quando o operando associado está em estado TRUE. Base de toda lógica combinacional em LAD.

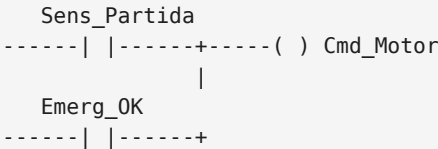
Quando usar: Leitura de sensores digitais (proximidade, pressostato, botoeira), habilitação de cadeias lógicas e condicionamento de blocos IEC. Em S7-1500, prefira tags otimizadas (%I, %Q) para acesso direto sem overhead de DB.

Sintaxe e Parâmetros: Operando (BOOL): Tag de entrada, memória M, bit de DB ou entrada física %I. O contato avalia o estado binário no início do ciclo de scan (process image input). Não possui pinos EN/ENO — é elemento de contato puro.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato NA AQUI]



STL (Statement List):

```
// Contato NA: instrução A (AND) / O (OR) conforme posição na rede
A    "Sens_Partida"
A    "Emerg_OK"
=    "Cmd_Motor"
```

Contato Normalmente Fechado (NF) — |/|

Nome e Objetivo: Elemento de entrada que conduz quando o operando está em FALSE. Inversão lógica de sinal de campo ou condição de segurança.

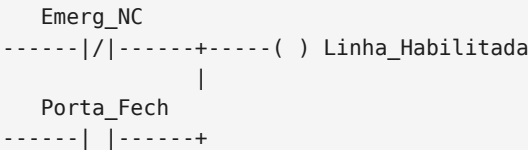
Quando usar: Intertravamentos de parada de emergência (NF em série), detecção de falta de sinal, lógica de alarme por perda de comunicação. Em STL, equivalente a AN (AND NOT).

Sintaxe e Parâmetros: Operando (BOOL): Mesmas regras do contato NA. A avaliação ocorre no ciclo atual; não há debounce nativo — implemente TON externo se necessário.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Contato NF AQUI]



STL (Statement List):

```
AN    "Emerg_NC"      // AND NOT – contato NF
A     "Porta_Fech"
=     "Linha_Habilitada"
```

Bobina de Atribuição — ()

Nome e Objetivo: Escreve o RLO atual no operando de saída a cada ciclo de scan. Comportamento volátil: se a condição de entrada cair, a saída retorna a FALSE imediatamente.

Quando usar: Comandos momentâneos, sinalização de estados transitórios, saídas que devem espelhar condição lógica sem memorização. Não utilizar para partida de motor sem intertravamento SR/RS.

Sintaxe e Parâmetros: Operando (BOOL): Saída %Q, memória M ou bit de DB. Em S7-1200/1500, bobinas em OBs de alta prioridade podem sobrepor saídas de OB1 — respeite hierarquia de OBs.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Bobina AQUI]

```

      Condicao_OK
-----| |----- ( ) Lamp_Ready

```

STL (Statement List):

```

A      "Condicao_OK"
=      "Lamp_Ready"

```

Set (S) e Reset (R) — Bobinas de Memorização

Nome e Objetivo: Instruções de coil que alteram o estado do operando para TRUE (S) ou FALSE (R) quando o RLO = 1, mantendo o estado entre ciclos. Ordem de varredura define prioridade em conflito.

Quando usar: Sequências simples, alarmes latched, reset de falhas. Em STL clássico S7-300/400, S e R na mesma rede: o último processado vence. Para prioridade explícita, use blocos RS/SR.

Sintaxe e Parâmetros: Operando (BOOL): Bit a memorizar. S: set dominante se escrito por último. R: reset dominante se escrito por último. Não usar S/R no mesmo operando em redes paralelas sem análise de prioridade.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Set/Reset AQUI]

```

      Btn_Start
-----| |----- (S) Motor_Run

      Btn_Stop
-----| |----- (R) Motor_Run

```

STL (Statement List):

```

// Reset dominante: R após S
A      "Btn_Start"
S      "Motor_Run"
A      "Btn_Stop"
R      "Motor_Run"

```

1.2 Memorização de Estado e Flip-Flops (SR / RS)

SR (Set Dominant) e RS (Reset Dominant)

Nome e Objetivo: Blocos de flip-flop para memorização de estado lógico. SR prioriza Set quando S1 e R estão simultaneamente TRUE (Q = 1). RS prioriza Reset (Q = 0). Garantem comportamento determinístico em comando concorrente.

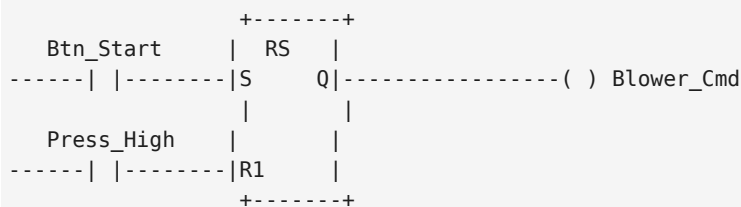
Quando usar: Partida/parada de motores, válvulas ON/OFF, intertravamentos de processo. RS obrigatório quando parada de emergência ou condição de segurança não pode ser sobreposta por partida. SR apenas quando Set deve prevalecer em conflito (ex.: modo automático forçado).

Sintaxe e Parâmetros: S1/S (BOOL): Condição de Set. R/R1 (BOOL): Condição de Reset. Q (BOOL): Estado memorizado. Instanciar como multi-instância em FB ou DB dedicado tipo SR/RS.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RS AQUI]



STL (Statement List):

```
// Reset dominante via ordem STL
A    "Btn_Start"
S    "Blower_Cmd"
A    "Press_High"
R    "Blower_Cmd"

// Ou via CALL do bloco IEC
CALL "RS_Blower", "DB_RS_Blower"
SET  := "Btn_Start"
RESET1 := "Press_High"
Q    := "Blower_Cmd"
```

1.3 Operações de Detecção de Borda (P_TRIG / N_TRIG / R_TRIG / F_TRIG)

P_TRIG (Positive Trigger) / FP em STL

Nome e Objetivo: Emite pulso TRUE por exatamente 1 ciclo de scan na transição 0→1 do sinal monitorado. R_TRIG é variante em bloco com saída Q e entrada CLK.

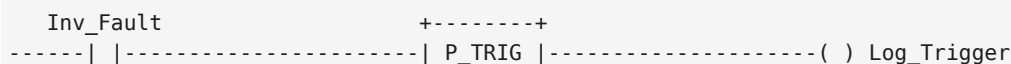
Quando usar: Disparo único de requisições (REQ em GET/PUT), incremento de contadores de evento, registro de alarmes sem repetição cíclica, handshake de comunicação Profinet.

Sintaxe e Parâmetros: CLK (BOOL): Sinal monitorado. M_BIT (BOOL): Memória de borda — exclusiva, estática em FB. Q (BOOL): Pulso de 1 scan. Em STL: FP "Edge_Mem" após A do sinal.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO P_TRIG AQUI]



```

      |      |
"Inst_DB".Edge_Mem_InvFault----| M_BIT |
      +-----+

```

STL (Statement List):

```

A      "Inv_Fault"
FP     "Inst_DB".Edge_Mem_InvFault
=      "Log_Trigger"

```

N_TRIG (Negative Trigger) / FN em STL

Nome e Objetivo: Pulso de 1 ciclo na transição 1→0. F_TRIG é equivalente em bloco.

Quando usar: Detecção de fim de ciclo, liberação de intertravamento após retorno de sensor, contagem de peças na saída da zona de detecção.

Sintaxe e Parâmetros: Mesmos parâmetros do P_TRIG. FN em STL requer memória de borda dedicada.

Exemplo Prático de Aplicação:**LAD (Ladder):**

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO N_TRIG AQUI]

```

      Sens_Peca      +-----+
-----| |-----| N_TRIG |----- ( ) Peca_Saida
      |      |
"DB_Prod".Edge_Mem_Sens-----| M_BIT |
      +-----+

```

STL (Statement List):

```

A      "Sens_Peca"
FN     "DB_Prod".Edge_Mem_Sens
=      "Peca_Saida"

```

R_TRIG / F_TRIG (Blocos IEC)

Nome e Objetivo: Blocos encapsulados de detecção de borda positiva (R_TRIG) e negativa (F_TRIG) com interface padronizada EN/CLK/Q.

Quando usar: FBs reutilizáveis, bibliotecas corporativas BLBW, padronização de código em equipes multidisciplinares.

Sintaxe e Parâmetros: CLK (BOOL): Entrada. Q (BOOL): Saída pulsada. Memória interna no DB de instância — não expor M_BIT externamente.

Exemplo Prático de Aplicação:**LAD (Ladder):**

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO R_TRIG AQUI]

```

      Start_Pulse    +-----+
-----| |-----| R_TRIG |
      |      |
      CLK      Q|----- ( ) Step_Advance
      +-----+

```

STL (Statement List):

```

CALL  "R_TRIG_Step", "DB_R_TRIG_Step"
      CLK := "Start_Pulse"
      Q   := "Step_Advance"

```

CAPÍTULO 2: Timer Operations (IEC Timers)

2.1 TON (Timer On-Delay)

TON — Generate On-Delay

Nome e Objetivo: Temporizador IEC com retardo na ativação. Q = TRUE somente após IN permanecer TRUE durante PT. Perda de IN zera ET e Q imediatamente.

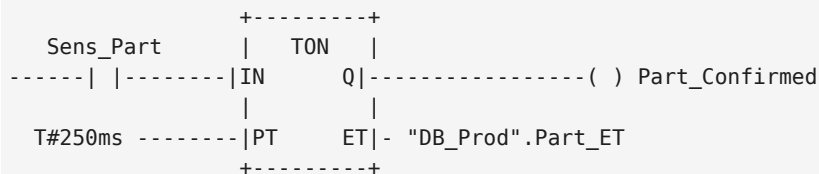
Quando usar: Debounce de sensores, sequenciamento de partidas escalonadas, retardo pós-rearme de emergência, confirmação de presença de peça.

Sintaxe e Parâmetros: IN (BOOL): Habilitação. PT (TIME): Preset T#250ms, T#5s, etc. Q (BOOL): Saída temporizada. ET (TIME): Tempo decorrido. DB: IEC_TIMER ou instância em FB.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TON AQUI]



STL (Statement List):

```
CALL "TON_Part_Instance"  
  IN := "Sens_Part"  
  PT := T#250MS  
  Q  := "Part_Confirmed"  
  ET := "DB_Prod".Part_ET
```

2.2 TOF (Timer Off-Delay)

TOF — Generate Off-Delay

Nome e Objetivo: Q assume TRUE instantaneamente com IN = TRUE. Ao cair IN, inicia contagem de PT antes de Q = FALSE. Retorno de IN durante contagem cancela e reinicia.

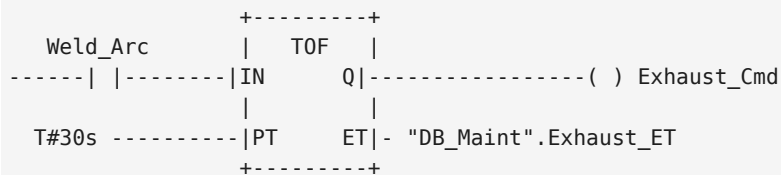
Quando usar: Ventilação pós-processo, lubrificação residual, extração de fumos após solda, despressurização com dreno temporizado.

Sintaxe e Parâmetros: IN, PT, Q, ET — idênticos ao TON. Tipo de timer: IEC_TIMER.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TOF AQUI]



STL (Statement List):

```
CALL "TOF_Exhaust_Instance"  
  IN := "Weld_Arc"
```

```
PT := T#30S
Q  := "Exhaust_Cmd"
ET := "DB_Maint".Exhaust_ET
```

2.3 TP (Timer Pulse)

TP — Generate Pulse

Nome e Objetivo: Gera pulso de duração fixa PT na borda de subida de IN. Q permanece TRUE por PT independente do estado de IN após disparo. Nova borda só é aceita após término do pulso.

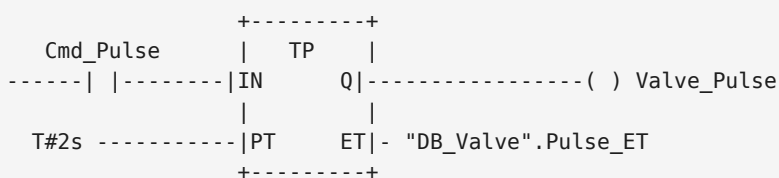
Quando usar: Impulso de válvula proporcional, blink de alarme, pulso de reset remoto, temporização de acionamento de relé monostável.

Sintaxe e Parâmetros: IN (BOOL): Flanco de disparo. PT (TIME): Duração do pulso. Q (BOOL): Saída pulsada. ET: tempo restante do pulso.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TP AQUI]



STL (Statement List):

```
CALL "TP_Valve_Instance"
  IN := "Cmd_Pulse"
  PT := T#2S
  Q  := "Valve_Pulse"
  ET := "DB_Valve".Pulse_ET
```

2.4 TONR (Time Accumulator)

TONR — Time Accumulator

Nome e Objetivo: Acumula tempo de IN = TRUE entre ciclos. R (BOOL) zera ET e Q. Diferente do TON, perda momentânea de IN preserva tempo acumulado.

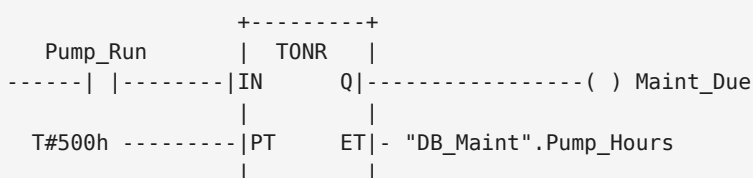
Quando usar: Horímetro de equipamento, manutenção preventiva por tempo de operação, controle de tempo máximo de ciclo acumulado, billing de tempo de bomba.

Sintaxe e Parâmetros: IN (BOOL): Contagem ativa. PT (TIME): Tempo alvo acumulado. R (BOOL): Reset. Q (BOOL): TRUE quando ET >= PT. ET (TIME): Tempo total acumulado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO TONR AQUI]



Reset_Hours		R	
-----	-----		
		+-----+	

STL (Statement List):

```
CALL  "TONR_Pump_Hours"
  IN  := "Pump_Run"
  PT  := T#500H
  R   := "Reset_Hours"
  Q   := "Maint_Due"
  ET  := "DB_Maint".Pump_Hours
```

CAPÍTULO 3: Counter Operations (IEC Counters)

3.1 CTU (Count Up)

CTU — Count Up

Nome e Objetivo: Contador incremental IEC. Incrementa CV em borda de subida de CU quando R = FALSE. Q = TRUE quando CV >= PV.

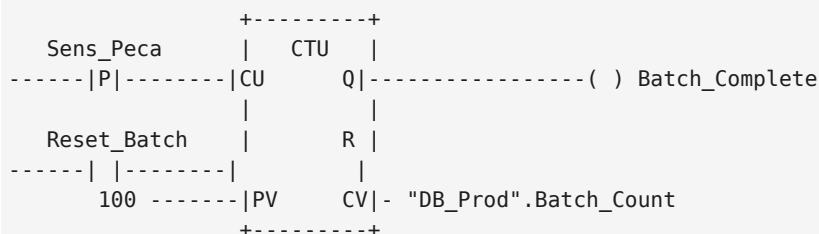
Quando usar: Contagem de peças produzidas, batching, limite de ciclos para manutenção, controle de paletização.

Sintaxe e Parâmetros: CU (BOOL): Count Up — borda de subida. R (BOOL): Reset CV para 0. PV (INT/DINT): Preset. CV (INT/DINT): Valor atual. Q (BOOL): CV >= PV.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTU AQUI]



STL (Statement List):

```

CALL "CTU_Batch"
CU := "Sens_Peca"
R := "Reset_Batch"
PV := 100
Q := "Batch_Complete"
CV := "DB_Prod".Batch_Count
  
```

3.2 CTD (Count Down)

CTD — Count Down

Nome e Objetivo: Contador decremental. CD em borda de subida decrementa CV. LD carrega PV em CV. Q = TRUE quando CV <= 0.

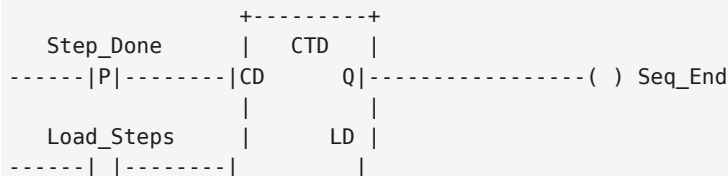
Quando usar: Contagem regressiva de etapas, dispensação de N doses, sequenciador com número fixo de passos restantes.

Sintaxe e Parâmetros: CD (BOOL): Count Down. LD (BOOL): Load — carrega PV em CV. PV, CV, Q conforme CTU. Q ativo em CV = 0.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTD AQUI]



```

10 -----|PV      CV|- "DB_Seq".Steps_Left
          +-----+

```

STL (Statement List):

```

CALL  "CTD_Sequence"
  CD := "Step_Done"
  LD := "Load_Steps"
  PV := 10
  Q  := "Seq_End"
  CV := "DB_Seq".Steps_Left

```

3.3 CTUD (Count Up/Down)

CTUD — Count Up/Down

Nome e Objetivo: Contador bidirecional. CU incrementa, CD decrementa. QU e QD indicam $CV \geq PV$ e $CV \leq 0$ respectivamente. R zera CV.

Quando usar: Controle de estoque em silos, posicionamento incremental, filas de buffer com entrada/saída simultânea.

Sintaxe e Parâmetros: CU, CD, R, LD, PV, CV, QU, QD. Prioridade de CU e CD simultâneos: conforme manual IEC — verificar versão de firmware.

Exemplo Prático de Aplicação:**LAD (Ladder):**

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CTUD AQUI]

```

          +-----+
Entry_Sens | CTUD |
-----|P|-----|CU    QU|----- ( ) Silo_Full
          |      |
Exit_Sens  |      |
-----|P|-----|CD    QD|----- ( ) Silo_Empty
          |      |
Reset_Level |      R  |
-----| |-----|      |
          1000 -----|PV    CV|- "DB_Silo".Level
          +-----+

```

STL (Statement List):

```

CALL  "CTUD_Silo_Level"
  CU := "Entry_Sens"
  CD := "Exit_Sens"
  R  := "Reset_Level"
  PV := 1000
  QU := "Silo_Full"
  QD := "Silo_Empty"
  CV := "DB_Silo".Level

```

CAPÍTULO 4: Comparator Operations

4.1 Operações Matemáticas Relacionais (==, <>, >=, <=, >, <)

CMP == / CMP <> / CMP >= / CMP <= / CMP > / CMP <

Nome e Objetivo: Comparação aritmética entre dois operandos do mesmo tipo numérico ou tempo. Resultado BOOL na saída do box ou como condição de contato em LAD.

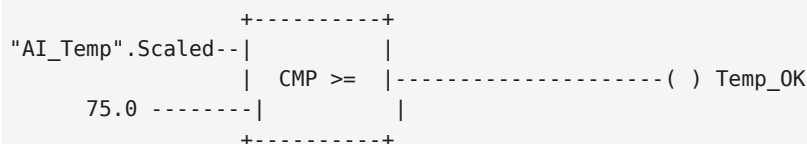
Quando usar: Setpoint de temperatura, comparação de posição encoder, verificação de receita, intertravamento por faixa de velocidade.

Sintaxe e Parâmetros: IN1, IN2: Operandos comparáveis (INT, DINT, REAL, TIME). Tipos devem coincidir ou usar CONVERT prévio. Saída BOOL integrada ao RLO em LAD.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CMP >= AQUI]



STL (Statement List):

```

L    "AI_Temp".Scaled
>=R  75.0
=    "Temp_OK"
  
```

4.2 Checagem de Limites (IN_Range / OUT_Range)

IN_Range / OUT_Range

Nome e Objetivo: IN_Range: TRUE se MIN <= IN <= MAX. OUT_Range: TRUE se IN < MIN ou IN > MAX. Simplifica faixas sem dupla comparação.

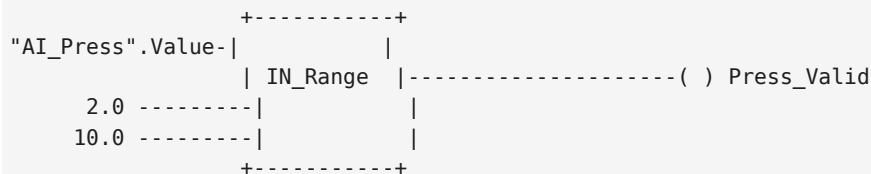
Quando usar: Validação de sinal analógico dentro de faixa operacional, detecção de sensor quebrado (4-20mA), tolerância de processo em estação SanBox BLBW.

Sintaxe e Parâmetros: IN (numeric): Valor avaliado. MIN, MAX: Limites inclusivos (IN_Range). Tipos homogêneos REAL recomendado para analógicos.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO IN_Range AQUI]



STL (Statement List):

```

CALL  "IN_Range_Press"
IN    := "AI_Press".Value
MIN   := 2.0
MAX   := 10.0
  
```

OUT := "Press_Valid"

CAPÍTULO 5: Math Functions

5.1 Cálculos Aritméticos Básicos (ADD, SUB, MUL, DIV)

ADD / SUB / MUL / DIV

Nome e Objetivo: Operações aritméticas IEC com EN/ENO. DIV com proteção: ENO = FALSE em divisão por zero — sempre monitorar ENO em cálculos críticos.

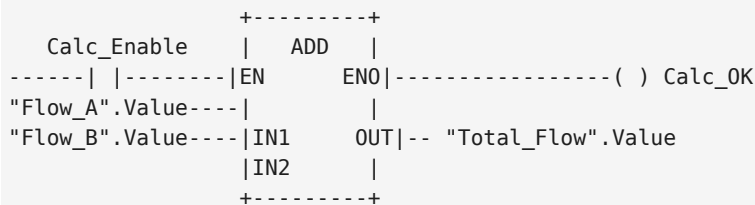
Quando usar: Cálculo de vazão, balanço massa, offset de setpoint, conversão de unidades em runtime.

Sintaxe e Parâmetros: EN (BOOL): Habilitação. IN1, IN2: Operandos. OUT: Resultado. ENO (BOOL): Sem erro. Tipos: INT, DINT, REAL — resultado conforme tipo maior.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ADD AQUI]



STL (Statement List):

```

A    "Calc_Enable"
CALL "ADD_Flow"
    IN1 := "Flow_A".Value
    IN2 := "Flow_B".Value
    OUT := "Total_Flow".Value
    ENO => "Calc_OK"
  
```

5.2 Seleção de Extremos e Limitação (MIN, MAX, LIMIT)

MIN / MAX / LIMIT

Nome e Objetivo: MIN: menor de IN1/IN2. MAX: maior. LIMIT: restringe IN entre MN e MX (MN se IN<MN, MX se IN>MX, senão IN).

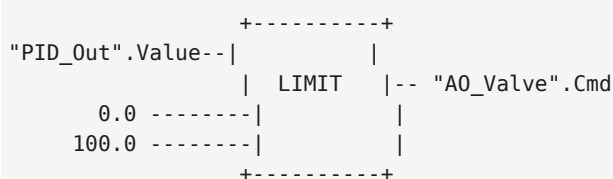
Quando usar: Saturação de comando analógico, seleção de menor pressão entre redundância, clamping de PID output, proteção de atuador.

Sintaxe e Parâmetros: IN1, IN2 (MIN/MAX). IN, MN, MX (LIMIT). OUT: resultado. Tipos REAL para analógicos.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO LIMIT AQUI]



STL (Statement List):

```
CALL "LIMIT_Valve"
  IN := "PID_Out".Value
  MN := 0.0
  MX := 100.0
  OUT := "AO_Valve".Cmd
```

5.3 Funções Trigonométricas e Exponenciais Avançadas

SIN / COS / TAN / SQRT / EXPT / LN / EXP

Nome e Objetivo: Funções matemáticas em REAL. Entrada e saída em radianos para trigonômicas. SQRT de valor negativo gera ENO = FALSE.

Quando usar: Cinemática de braço robótico, cálculo de coordenadas em esteira circular, escalonamento não linear, dosing exponencial.

Sintaxe e Parâmetros: IN (REAL): Operando. OUT (REAL): Resultado. ENO: status. CALCULATE disponível para expressões compostas.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SQRT AQUI]

```

      +-----+
"Tank_Vol".m3-----|  SQRT  |-- "Level_Est".m
      +-----+
```

STL (Statement List):

```
CALL "SQRT_Level"
  IN := "Tank_Vol".m3
  OUT := "Level_Est".m
```

CAPÍTULO 6: Move Operations

6.1 Transferência de Bloco Escalar (MOVE)

MOVE — Move Value

Nome e Objetivo: Cópia de valor entre tags de tipos compatíveis. Suporta BOOL até estruturas e VARIANT conforme versão. Substituto moderno de BLK MOV e transferências STL.

Quando usar: Cópia de setpoint recebido via comunicação, transferência de estrutura de receita, inicialização de parâmetros de FB.

Sintaxe e Parâmetros: EN, ENO. IN: origem. OUT: destino. Tipos devem ser atribuíveis — usar CONVERT se necessário.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MOVE AQUI]

```

      +-----+
      | Update_Recipe | MOVE |
      +-----+-----+
      | "HMI".Setpoint | IN   | ENO |
      +-----+-----+
      | "DB_Recipe".Setpoint | OUT |
      +-----+

```

STL (Statement List):

```

A      "Update_Recipe"
CALL   "MOVE_Setpoint"
      IN  := "HMI".Setpoint
      OUT := "DB_Recipe".Setpoint

```

6.2 Manipulação em Lote e Serialização (MOVE_BLK, FILL_BLK, SWAP)

MOVE_BLK / FILL_BLK / SWAP

Nome e Objetivo: MOVE_BLK: copia N elementos de array. FILL_BLK: preenche array com valor constante. SWAP: inverte ordem de bytes (endianness) em WORD/DWORD.

Quando usar: Buffer de comunicação, cópia de zona de DB, inicialização de arrays de tendência, conversão Modbus RTU (byte swap).

Sintaxe e Parâmetros: MOVE_BLK: SRC, DST, COUNT, INDEX. FILL_BLK: SRC (valor), DST, COUNT. SWAP: IN/OUT WORD/DWORD.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MOVE_BLK AQUI]

```

      +-----+
      | Copy_Enable | MOVE_BLK |
      +-----+-----+
      | "Src_Array" | IN       | ENO |
      +-----+-----+
      | "Dst_Array" |          |
      | 20          | COUNT   |
      +-----+-----+

```

STL (Statement List):

```

CALL   "MOVE_BLK_Data"
      SRC_ARRAY := "Src_Array"

```



```
DST_ARRAY := "Dst_Array"
COUNT   := 20
```

Serialize / Deserialize

Nome e Objetivo: Serializa estrutura tipada em array de BYTE para transmissão. Deserialize reconstrói estrutura a partir do buffer.

Quando usar: Telemetria SanBox BLBW, envio de struct via PUT/GET, logging compacto em CPU sem cartão SD.

Sintaxe e Parâmetros: SRC/DST: VARIANT ou struct. POS: offset no buffer. BUFFER: ARRAY OF BYTE.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO Serialize AQUI]

```

      Tx_Enable      +-----+
                    | Serialize |
-----| |-----|EN      ENO|
"DB_Telemetry"----|          |
"Tx_Buffer"-----|          |
                    +-----+
```

STL (Statement List):

```
CALL "Serialize_Telemetry"
SRC   := "DB_Telemetry"
BUFFER := "Tx_Buffer"
POS   := 0
```

CAPÍTULO 7: Conversion Operations

7.1 Conversão Explícita de Cast (CONVERT)

CONVERT

Nome e Objetivo: Conversão tipada explícita entre tipos numéricos (INT↔REAL↔DINT↔DWORD). Overflow gera ENO = FALSE em alguns casos — validar.

Quando usar: Processamento de AI (INT para REAL), preparação de operandos para CMP e math, integração com drives (DWORD).

Sintaxe e Parâmetros: IN: tipo origem. OUT: tipo destino declarado no box. ENO: status.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO CONVERT AQUI]

```

      +-----+
"AI_Raw".Int-----|  CONVERT  |-- "AI_Raw".Real
      +-----+

```

STL (Statement List):

```

CALL  "CONVERT_AI"
  IN   := "AI_Raw".Int
  OUT  := "AI_Raw".Real

```

7.2 Arredondamento e Truncamento (ROUND, CEIL, FLOOR, TRUNC)

ROUND / CEIL / FLOOR / TRUNC

Nome e Objetivo: ROUND: arredondamento matemático para inteiro. CEIL: próximo inteiro superior. FLOOR: próximo inferior. TRUNC: descarta parte fracionária.

Quando usar: Exibição em IHM sem decimal, cálculo de quantidade inteira de embalagens, indexação de array por REAL.

Sintaxe e Parâmetros: IN (REAL/LREAL). OUT (INT/DINT conforme bloco).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ROUND AQUI]

```

      +-----+
"Dose_Kg".Real----|  ROUND  |-- "Dose_Kg".Int
      +-----+

```

STL (Statement List):

```

CALL  "ROUND_Dose"
  IN   := "Dose_Kg".Real
  OUT  := "Dose_Kg".Int

```

7.3 Escalonamento de Sinais Analógicos (SCALE_X, NORM_X)

SCALE_X / NORM_X

Nome e Objetivo: NORM_X: normaliza entrada de faixa RAW (ex. 0-27648) para 0.0-1.0. SCALE_X: aplica ganho e offset de 0-1 para engenharia (0-10 bar). Par inverso para escrita em AO.

Quando usar: Padronização de tratamento AI/AO em biblioteca BLBW, eliminação de fórmulas manuais em cada tag analógica.

Sintaxe e Parâmetros: NORM_X: MIN, VALUE, MAX → OUT 0..1. SCALE_X: MIN, VALUE(0..1), MAX → engenharia.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SCALE_X AQUI]

```

+-----+
"AI_Raw".Value----|  NORM_X  |-- "AI_Norm".Pct
    0  -----|          |
 27648 -----|          |
+-----+

+-----+
"AI_Norm".Pct----|  SCALE_X  |-- "AI_Press".Bar
    0.0 -----|          |
   10.0 -----|          |
+-----+

```

STL (Statement List):

```

CALL  "NORM_X_AI"
  MIN   := 0
  VALUE := "AI_Raw".Value
  MAX   := 27648
  OUT   := "AI_Norm".Pct

CALL  "SCALE_X_Press"
  MIN   := 0.0
  VALUE := "AI_Norm".Pct
  MAX   := 10.0
  OUT   := "AI_Press".Bar

```

CAPÍTULO 8: Program Control Operations

8.1 Saltos e Roteamento Lógico (JMP, JMPN, LABEL, SWTCH)

JMP / JMPN / LABEL

Nome e Objetivo: JMP: salta para LABEL se RLO=1. JMPN: salta se RLO=0. LABEL: destino do salto. Código entre JMP e LABEL não é processado quando salto ativo.

Quando usar: Otimização de scan em OB1 extensa, bypass de diagnóstico, desabilitação condicional de redes STL. Usar com moderação — prejudica rastreabilidade.

Sintaxe e Parâmetros: LABEL: identificador único na rede. JMP/JMPN: coil-style em LAD. Apenas dentro do mesmo OB/FC.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO JMP AQUI]

      Skip_Diag
-----| |----- (JMP) Label_Prod

      // Redes de diagnóstico omitidas quando Skip_Diag = TRUE

                                LABEL
      Label_Prod:
      Process_Run
-----| |----- ( ) Line_Run
```

STL (Statement List):

```
A      "Skip_Diag"
JMP     Label_Prod
// ... diagnóstico ...
Label_Prod: A      "Process_Run"
=       "Line_Run"
```

SWTCH (Jump Distributor)

Nome e Objetivo: Distribuidor de salto indexado. K (INT) seleciona qual saída CASE é ativada, roteando fluxo para múltiplos LABELs.

Quando usar: Sequenciadores em STL puro, máquinas de estados em FC sem GRAPH, seleção de rotina por código de receita.

Sintaxe e Parâmetros: K (INT): índice 1..n. Entradas CASE1..CASEn com LABEL associado.

Exemplo Prático de Aplicação:

LAD (Ladder):

```
[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SWTCH AQUI]

      +-----+
"Recipe".ID-----|  SWTCH  |
                  | K:=ID   |
                  | CASE1   |----> Label_R1
                  | CASE2   |----> Label_R2
                  +-----+
```

STL (Statement List):

```
L      "Recipe".ID
```

```
SWTCH  
CASE1 Label_R1  
CASE2 Label_R2
```

8.2 Finalização de Bloco (RET)

RET — Return

Nome e Objetivo: Encerra execução do OB/FC/Fb atual imediatamente. Redes subsequentes não são processadas neste ciclo.

Quando usar: FC de comunicação com abort em erro, proteção contra execução sem habilitação, saída antecipada em diagnóstico.

Sintaxe e Parâmetros: RET como coil quando RLO=1. Em STL: RET condicionado ou incondicional.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO RET AQUI]

```
      Comm_Error  
-----| |----- (RET)
```

STL (Statement List):

```
A      "Comm_Error"  
RET
```

CAPÍTULO 9: Word Logic Operations

9.1 Mascaramento a Nível de Palavra (AND, OR, XOR)

AND / OR / XOR (Word)

Nome e Objetivo: Operação bit a bit entre WORD/DWORD. AND: máscara de bits ativos. OR: união. XOR: comparação e toggle.

Quando usar: Máscara de status de drive em DWORD, extração de nibble, diagnóstico de palavra de falha Profinet, reset seletivo de bits.

Sintaxe e Parâmetros: IN1, IN2 (WORD/DWORD). OUT: resultado mesma largura.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO AND Word AQUI]

```

      +-----+
"Drive".Status----|   AND   |-- "Drive".Masked
16#00FF -----|   |
      +-----+
```

STL (Statement List):

```

L    "Drive".Status
L    16#00FF
AND
T    "Drive".Masked
```

9.2 Manipulação Estrutural (ENCO, DECO, SEL, MUX, DEMUX)

ENCO / DECO

Nome e Objetivo: ENCO: converte posição de bit TRUE em INT (encoder one-hot). DECO: converte INT em word com bit correspondente em TRUE.

Quando usar: Seleção de canal ativo, decodificador de posição em matriz de válvulas, interface com hardware one-hot.

Sintaxe e Parâmetros: ENCO: IN (WORD), OUT (INT bit position). DECO: IN (INT), OUT (WORD).

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ENCO AQUI]

```

      +-----+
"Valve_Mask".Word-|  ENCO  |-- "Valve_Active".Idx
      +-----+
```

STL (Statement List):

```

CALL  "ENCO_Valve"
  IN   := "Valve_Mask".Word
  OUT  := "Valve_Active".Idx
```

SEL / MUX / DEMUX

Nome e Objetivo: SEL: se G=0 OUT=IN0, se G=1 OUT=IN1. MUX: seleciona entre N entradas por índice K. DEMUX: distribui entrada para uma de N saídas por K.

Quando usar: Seletor manual/auto de setpoint, multiplexação de sensores redundantes, roteamento de comando para um de vários atuadores.

Sintaxe e Parâmetros: SEL: G, IN0, IN1. MUX: K (INT), IN0..INn. DEMUX: K, IN, OUT0..OUTn.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO MUX AQUI]

```

      +-----+
"Mode".Manual-----|   MUX   |-- "Setpoint".Active
"HMI".SP_Man-----|         |
"DCS".SP_Auto-----|         |
"Mode".SelIdx-----|   K     |
      +-----+
```

STL (Statement List):

```
CALL  "MUX_Setpoint"
      K    := "Mode".SelIdx
      IN0  := "HMI".SP_Man
      IN1  := "DCS".SP_Auto
      OUT  := "Setpoint".Active
```

CAPÍTULO 10: Shift and Rotate Operations

10.1 Deslocamento de Bits (SHR, SHL)

SHR / SHL

Nome e Objetivo: SHR: desloca bits para direita, bits baixos perdem, entrada N define quantidade. SHL: desloca à esquerda. Útil para divisão/multiplicação por potência de 2.

Quando usar: Serialização manual de bits, conversão de formato de encoder, alinhamento de dados em buffer, otimização aritmética em INT.

Sintaxe e Parâmetros: IN (WORD/DWORD). N (INT): posições. OUT: resultado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO SHR AQUI]

```
+-----+
"Reg".Data-----|  SHR  |-- "Reg".Shifted
      2 -----|  N    |
+-----+
```

STL (Statement List):

```
CALL  "SHR_Reg"
  IN   := "Reg".Data
  N    := 2
  OUT  := "Reg".Shifted
```

10.2 Rotação de Registradores (ROR, ROL)

ROR / ROL

Nome e Objetivo: Rotação circular de bits — bits que saem reentram do lado oposto. Preserva todos os bits sem perda.

Quando usar: CRC bit-a-bit, criptografia leve de token, simulação de registrador de deslocamento em hardware, algoritmos de hash industrial.

Sintaxe e Parâmetros: IN (WORD/DWORD). N (INT): posições de rotação. OUT: resultado.

Exemplo Prático de Aplicação:

LAD (Ladder):

[INSERIR IMAGEM DO ÍCONE DA FUNÇÃO ROR AQUI]

```
+-----+
"Token".Word-----|  ROR  |-- "Token".Rotated
      1 -----|  N    |
+-----+
```

STL (Statement List):

```
CALL  "ROR_Token"
  IN   := "Token".Word
  N    := 1
  OUT  := "Token".Rotated
```